

WEEK 8 GIT

HANDS ON1

Step 1: Setup your machine with Git Configuration

```
swethaganesh@swetha MINGW64 ~
$ git --version
git version 2.42.0.windows.1

swethaganesh@swetha MINGW64 ~
$ cd ~

swethaganesh@swetha MINGW64 ~
$ mkdir -p ~/git-hol-practice

swethaganesh@swetha MINGW64 ~
$ cd ~/git-hol-practice

swethaganesh@swetha MINGW64 ~/git-hol-practice
$ mkdir GitDemo

swethaganesh@swetha MINGW64 ~/git-hol-practice
$ cd GitDemo

swethaganesh@swetha MINGW64 ~/git-hol-practice/G
itDemo
$ git init -b master
Initialized empty Git repository in C:/Users/swe
thaganesh/git-hol-practice/GitDemo/.git/

swethaganesh@swetha MINGW64 ~/git-hol-practice/G
itDemo (master)
$ git config user.name "swethag"

swethaganesh@swetha MINGW64 ~/git-hol-practice/G
itDemo (master)
$ git config user.name "Swethag05"

swethaganesh@swetha MINGW64 ~/git-hol-practice/G
itDemo (master)
$ git config user.email "swethag1805@gmail.com"

swethaganesh@swetha MINGW64 ~/git-hol-practice/G
itDemo (master)
$ git config --list --local
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Swethag05
user.email=swethag1805@gmail.com
```

Step 2: Integrate notepad++.exe to Git and make it a default editor

```
wethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
git config core.editor "notepad"

wethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
notepad welcome.txt

wethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
git add welcome.txt

wethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
git commit
int: Waiting for your editor to close the file.
. unix2dos: converting file C:/Users/swethaganesh/git-hol-practice/GitDemo/.git/COMMIT_EDITMSG to DOS format...
os2unix: converting file C:/Users/swethaganesh/git-hol-practice/GitDemo/.git/COMMIT_EDITMSG to nix format...
master (root-commit) 4b46149] Add welcome.txt
1 file changed, 2 insertions(+)
create mode 100644 welcome.txt
```

Step 3: Add a file to source code repository

```
swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git remote add origin https://gitlab.com/gitdemo3851832/GitDemo.git

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git remote -v
origin https://gitlab.com/gitdemo3851832/GitDemo.git (fetch)
origin https://gitlab.com/gitdemo3851832/GitDemo.git (push)

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git push -u origin master
info: please complete authentication in your browser...

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 307 bytes | 307.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/gitdemo3851832/GitDemo.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ notepad welcome.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

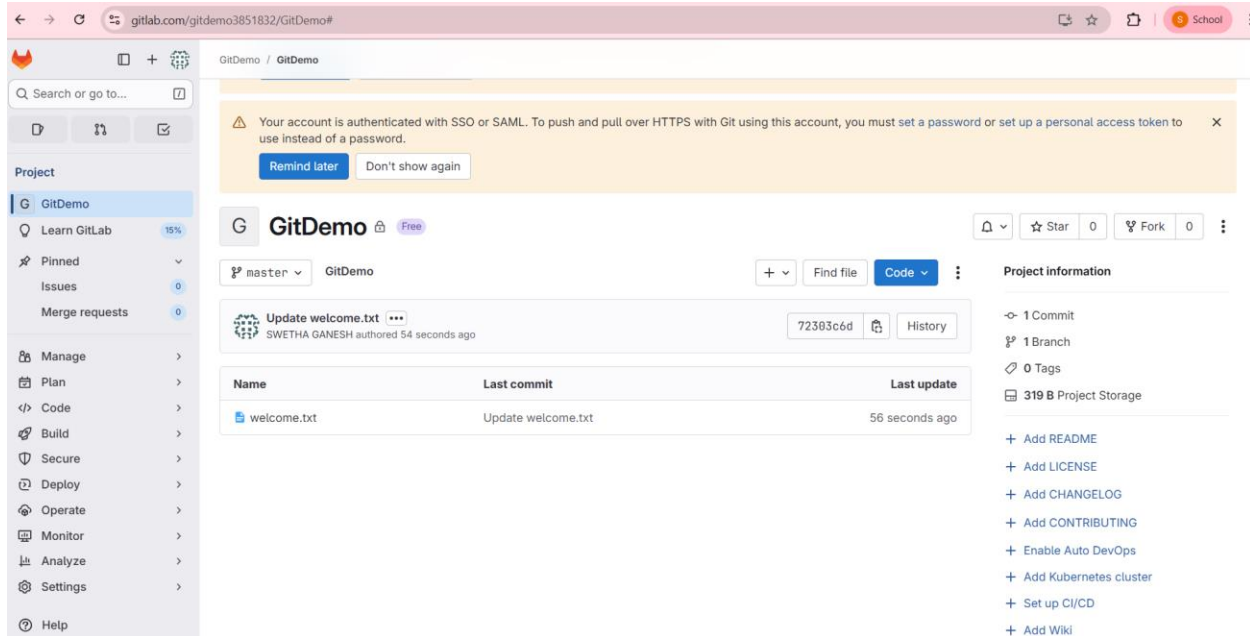
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   welcome.txt

no changes added to commit (use "git add" and/or "git commit -a")

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git add welcome.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git commit
hint: Waiting for your editor to close the file... unix2dos: converting file C:/Users/swethaganesh/git-hol-practice/GitDemo/.git/COMMIT_EDITMSG to DOS format.
dos2unix: converting file C:/Users/swethaganesh/git-hol-practice/GitDemo/.git/COMMIT_EDITMSG to Unix format...
[master 72303c6] Update welcome.txt
1 file changed, 3 insertions(+)

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 352 bytes | 352.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
```



HANDS ON2

Create a **“.log”** file and a **log folder** in the working directory of Git. Update the **.gitignore** file in such a way that on committing, these files (.log extensions and log folders) are ignored.

Verify if the git status reflects the same about working directory, local repository and git repository.

```
swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ echo "This is a test log file" > error.log

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ mkdir log

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ echo "Log folder test file" > log/test.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    error.log
    log/

nothing added to commit but untracked files present (use "git add" to track)

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ notepad .gitignore

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

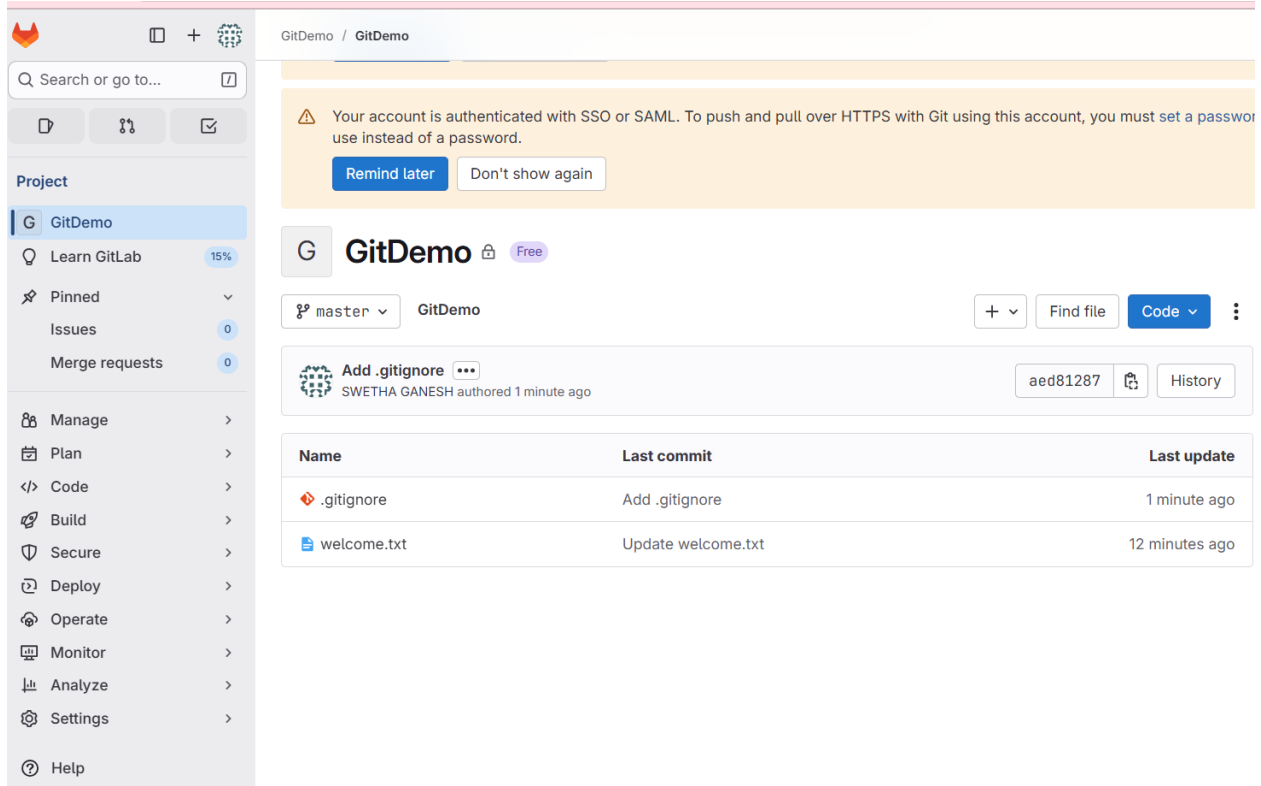
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore

nothing added to commit but untracked files present (use "git add" to track)

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git add .gitignore

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git commit
hint: Waiting for your editor to close the file... unix2dos: converting file C:/Us
dos2unix: converting file C:/Users/swethaganesh/git-hol-practice/GitDemo/.git/COMM
[master aed8128] Add .gitignore
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 329 bytes | 329.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/gitdemo3851832/GitDemo.git
 72303c6..aed8128  master -> master
```



HandsOn 3

Branching in Git

Definition:

Branching means creating a **separate line of development** in a Git repository so you can work on new features, fixes, or experiments without affecting the main code.

Key points:

- The default branch is usually master or main.
- You can create multiple branches to work independently.
- Changes in one branch do not affect other branches until you **merge** them.

Merging in Git

Definition:

Merging means combining changes from one branch into another branch.

Key points:

- Usually, you merge a feature branch into master after development and testing.
- Git tries to merge automatically, but if the same file/lines were changed in both branches, **merge conflicts** can happen.

Creating a Branch Request in GitLab

In GitLab, creating a branch request means **creating a new branch in the remote repository** (often done in the web interface or pushed from local).

```
swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git branch GitNewBranch

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git branch -a
  GitNewBranch
* master
  remotes/origin/master

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git checkout GitNewBranch
Switched to branch 'GitNewBranch'

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ echo "This is a file in GitNewBranch" > branchfile.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ mkdir branchfolder

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ echo "This is inside branchfolder" > branchfolder/info.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ git add branchfile.txt branchfolder/info.txt
warning: in the working copy of 'branchfile.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'branchfolder/info.txt', LF will be replaced by CRLF the next time Git touches it

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ git commit -m "Add branchfile.txt and branchfolder in GitNewBranch"
[GitNewBranch dbd6f8e] Add branchfile.txt and branchfolder in GitNewBranch
 2 files changed, 2 insertions(+)
 create mode 100644 branchfile.txt
 create mode 100644 branchfolder/info.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean
```

Creating a Merge Request in GitLab

A **Merge Request (MR)** in GitLab is a request to merge one branch into another — it's how teams review and approve changes before merging.

```
swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitNewBranch)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git diff master GitNewBranch
diff --git a/branchfile.txt b/branchfile.txt
new file mode 100644
index 0000000..70ab14d
--- /dev/null
+++ b/branchfile.txt
@@ -0,0 +1 @@
+This is a file in GitNewBranch
diff --git a/branchfolder/info.txt b/branchfolder/info.txt
new file mode 100644
index 0000000..526ddc5
--- /dev/null
+++ b/branchfolder/info.txt
@@ -0,0 +1 @@
+This is inside branchfolder

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git merge GitNewBranch
Updating aed8128..dbd6f8e
Fast-forward
 branchfile.txt      | 1 +
 branchfolder/info.txt | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 branchfile.txt
 create mode 100644 branchfolder/info.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git log --oneline --graph --decorate
* dbd6f8e (HEAD -> master, GitNewBranch) Add branchfile.txt and branchfolder in GitNewBranch
* aed8128 (origin/master) Add .gitignore
* 72303c6 Update welcome.txt
* 4b46149 Add welcome.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was dbd6f8e).

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```


HANDSONS 4:

What is a Merge Conflict?

A merge conflict happens when Git cannot automatically combine changes from two branches because the same part of the same file has been modified differently in each branch.

Git then stops the merge and marks the file so you can fix it manually.

When Do Conflicts Happen?

- Two branches change the same lines in a file.
- One branch deletes a file while another edits it.
- Changes to file structure or renaming files in different ways.

Resolving the conflict during merge

- Start merge → conflict occurs.
- Check with git status.
- Edit conflicted files → remove markers.
- git add the resolved files.
- git commit to finish merge.

```
swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitWork)
$ echo "<message>Hello from GitWork branch</message>" > hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitWork)
$ echo "<message>Hello again from GitWork branch</message>" > hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitWork)
$ git status
On branch GitWork
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.xml

nothing added to commit but untracked files present (use "git add" to track)
```

```

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitWork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitWork)
$ git commit -m "Add hello.xml in GitWork branch"
[GitWork bf41242] Add hello.xml in GitWork branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (GitWork)
$ git checkout master
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ echo "<message>Hello from Master branch</message>" > hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git commit -m "Add hello.xml in Master branch"
[master 4eb45cd] Add hello.xml in Master branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git log --oneline --graph --decorate --all
* 4eb45cd (HEAD -> master) Add hello.xml in Master branch
| * bf41242 (GitWork) Add hello.xml in GitWork branch
|/
* dbd6f8e Add branchfile.txt and branchfolder in GitNewBranch
* aed8128 (origin/master) Add .gitignore
* 72303c6 Update welcome.txt
* 4b46149 Add welcome.txt

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git diff master GitWork
diff --git a/hello.xml b/hello.xml
index 7feff86..31b34ee 100644
--- a/hello.xml
+++ b/hello.xml
@@ -1,1 @@
- <message>Hello from Master branch</message>
+ <message>Hello again from GitWork branch</message>

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git merge GitWork
Auto-merging hello.xml
CONFLICT (add/add): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.

```

```

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master|MERGING)
$ notepad hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master|MERGING)
$ git add hello.xml

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master|MERGING)
$ git commit -m "Merge GitWork into master and resolve conflict"
[master c0a1747] Merge GitWork into master and resolve conflict

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ cp hello.xml hello.xml~

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ echo "*~" >> .gitignore

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git commit -m "Ignore backup files"
[master 24993ee] Ignore backup files
1 file changed, 1 insertion(+)

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git branch
  GitWork
* master

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git branch -d GitWork
Deleted branch GitWork (was bf41242).

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git log --oneline --graph --decorate
* 24993ee (HEAD -> master) Ignore backup files
* c0a1747 Merge GitWork into master and resolve conflict
| \
| * bf41242 Add hello.xml in GitWork branch
* | 4eb45cd Add hello.xml in Master branch
| /
* dbd6f8e Add branchfile.txt and branchfolder in GitNewBranch
* aed8128 (origin/master) Add .gitignore
* 72303c6 Update welcome.txt
* 4b46149 Add welcome.txt

```

HANDSON 5

Why Clean Up?

Over time, a repository may contain:

- **Untracked files** (created locally but never staged)
- **Ignored files** (temporary files like .log, ~ backups, node_modules, etc.)
- **Stale branches** that are no longer needed

Cleaning up keeps the working directory organized, prevents accidental commits, and ensures only relevant files are pushed to the remote.

Command	Purpose
git clean -f	Remove untracked files
git clean -fd	Remove untracked files and folders
git clean -fx	Remove ignored + untracked files
git branch -d branch	Delete merged local branch
git push origin branch	Push branch changes to remote
git push origin --delete branch	Delete branch in remote repo

```

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git pull origin master
From https://gitlab.com/gitdemo3851832/GitDemo
* branch                master      -> FETCH_HEAD
Already up to date.

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git push origin master
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 4 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (17/17), 1.51 KiB | 772.00 KiB/s, done.
Total 17 (delta 5), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/gitdemo3851832/GitDemo.git
   aed8128..24993ee  master -> master

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$ git push origin --delete old-branch-name
error: unable to delete 'old-branch-name': remote ref does not exist
error: failed to push some refs to 'https://gitlab.com/gitdemo3851832/Gi

swethaganesh@swetha MINGW64 ~/git-hol-practice/GitDemo (master)
$

```



master GitDemo

Find file Code

Ignore backup files
SWETHA GANESH authored 6 minutes ago

24993ee9

History

Name	Last commit	Last update
branchfolder	Add branchfile.txt and branchfolder in ...	17 minutes ago
.gitignore	Ignore backup files	6 minutes ago
branchfile.txt	Add branchfile.txt and branchfolder in ...	17 minutes ago
<u>hello.xml</u> hello.xml	Merge GitWork into master and resolve...	6 minutes ago
welcome.txt	Update welcome.txt	37 minutes ago